

TinyNPU Foundations

Technical Onboarding Document

AI Accelerator Architecture for RTL Engineers

Project: TinyNPU — Ultra-Low-Power Configurable AI Accelerator for Real-Time Edge Vision

Verification target: Vivado simulation only (no FPGA board bring-up)

Prepared as project onboarding material — read before RTL work begins

Table of Contents

This document is written for an RTL engineer who is comfortable with SystemVerilog and Vivado but new to AI accelerator architecture specifically. It covers the concepts needed before writing PE-level RTL, organized to build intuition first, then formalize it with math, then connect it to concrete hardware structures.

Read it in order — each section builds on the last.

1. Why AI Accelerators Look the Way They Do

A CPU executes one instruction on a few operands per cycle. A GPU executes thousands of threads doing the same instruction on different data (SIMT). Neither is a great fit for CNN inference, because a CNN's compute pattern has a very specific shape:

- Massive data reuse. The same weight is multiplied against many different input pixels (weight reuse). The same input pixel is multiplied against many different filters (activation reuse). The same partial sum is accumulated across many multiply-adds (accumulator reuse).
- Regular, statically-known structure. A CNN layer's loop bounds (output height, width, channels, kernel size) are known before you run it — you can build hardware specialized for exactly this loop nest instead of a general-purpose ALU.
- Low precision tolerance. Neural nets tolerate INT8 or even INT4 arithmetic with small accuracy loss, giving 4x the compute density per unit area versus FP32, and much lower power per MAC.

The consequence: an AI accelerator is fundamentally an array of multiply-accumulate (MAC) units wired together to maximize data reuse and minimize movement of data to/from memory. Everything else in the design (buffers, controllers, DMA) exists to keep that MAC array fed without stalling.

The single most important idea in accelerator design: data movement costs more energy than compute.

This is the number every accelerator architect has memorized (from Horowitz's well-known ISSCC 2014 energy table, still broadly true in modern nodes):

Operation (45nm, illustrative)	Relative Energy
INT8 add	1x
INT8 multiply	~6x
Read 32-bit word from register file	~1x
Read 32-bit word from on-chip SRAM (small)	~5x
Read 32-bit word from on-chip SRAM (large, cross-die)	~25x
Read 32-bit word from off-chip DRAM	~200x

Takeaway: every architectural decision in this project — systolic array style, buffer sizing, dataflow — is really a decision about how many times a piece of data gets moved, because that dominates power far more than the MAC itself. This is why memory hierarchy work matters as much as PE design.

2. Quantization: Why and How

2.1 Why Quantize

Neural networks are trained in FP32. Running FP32 in hardware costs wider datapaths (32-bit multipliers are ~9x the area of 8-bit multipliers), more SRAM for the same tensor, and more power per operation. INT8 quantization keeps the trained weights' relative relationships while representing them with 8 bits.

2.2 The Math

A real value r is mapped to an integer q via:

$$r = S * (q - Z)$$

- S (scale): a positive float, maps the integer step size to the real range.
- Z (zero-point): an integer, the quantized value that represents real 0.0.

For a tensor with a known min/max range $[r_{\min}, r_{\max}]$ and n -bit integers (range $[q_{\min}, q_{\max}]$):

$$S = (r_{\max} - r_{\min}) / (q_{\max} - q_{\min})$$
$$Z = \text{round}(q_{\min} - r_{\min} / S)$$

Symmetric quantization (common for weights): forces $Z = 0$, range is $[-127, 127]$ for INT8. Simpler hardware (no zero-point subtraction needed before the multiply), slightly less range utilization.

Asymmetric quantization (common for activations, especially post-ReLU where values are ≥ 0): uses a nonzero Z , better range utilization, but requires a zero-point correction term in the accumulation math.

2.3 What Actually Happens in the MAC Datapath

For a quantized matmul $Y = W @ X$, expanding the dequantization gives you (this is the actual math the PE and accumulator need to implement):

$$Y_{\text{real}} \approx S_w * S_x * \sum [(W_q - Z_w)(X_q - Z_x)]$$

Expanding the product term:

$$\sum (W_q * X_q) - Z_x * \sum (W_q) - Z_w * \sum (X_q) + N * Z_w * Z_x$$

This is why real accelerators (e.g., Google's gemmlowp, the basis for TFLite's quantized kernels) precompute the $Z_w * \sum(X_q)$ and $N * Z_w * Z_x$ correction terms outside the inner MAC loop — the systolic array itself only ever computes the raw $\sum(W_q * X_q)$ term, and a separate, much smaller post-processing stage (often fused into the Accumulator or Activation Unit module) applies the correction, rescale ($S_w * S_x$), and re-quantization back to INT8 for the next layer.

Design implication for TinyNPU: if we use symmetric quantization for both weights and activations ($Z_w = Z_x = 0$), all correction terms vanish and the PE only ever computes plain $\sum(W_q * X_q)$ — dramatically simpler hardware, at a small cost in dynamic range utilization for post-ReLU activations. Most small edge accelerators make exactly this trade: symmetric INT8 everywhere. Recommended for TinyNPU Phase 3.

2.4 Requantization ("the rescale step")

After accumulating INT32 partial sums, you must convert back to INT8 before the next layer. This step is where BatchNorm fusion also happens (see Section 6). The general per-channel requantization is:

```
INT32_accum → multiply by (S_w * S_x / S_out) → round → clamp to INT8 range
```

Since $S_w * S_x / S_{out}$ is a real number (not a nice power of two), hardware implementations avoid floating point by representing it as a fixed-point multiplier + right-shift pair, computed offline during the quantization step of the Python reference model:

```
multiplier, shift = decompose_to_fixed_point(S_w * S_x / S_out)
result = (accum * multiplier) >> shift
```

This "integer-only multiply + shift" trick is exactly how TFLite's reference quantized kernels and most commercial NPUs do requantization without ever touching a float unit in hardware. The accumulator/activation module needs an integer multiplier and a barrel shifter — nothing more.

3. Systolic Arrays

3.1 What "Systolic" Actually Means

A systolic array is a grid of PEs where data moves rhythmically between neighbors on every clock cycle — like blood being pumped through the body (hence "systolic," coined by H.T. Kung in 1982). No PE talks to global memory directly during the compute burst; it only exchanges data with its immediate neighbors (north/south/east/west). This is what makes the array scalable: wiring is local, so the design doesn't hit a routing wall as the array grows, and it maps beautifully onto an FPGA's regular fabric.

3.2 The Three Canonical Dataflows

These describe what stays stationary in each PE while everything else streams through:

Weight-Stationary (WS)

- Each PE holds one weight value for the duration of a layer's computation.
- Activations stream in from one direction, partial sums propagate through/out the other.
- Best when: weight reuse is high (same filter applied at many spatial positions — true for conv layers with large feature maps).
- Cost: activations and partial sums both need to move every cycle.

Output-Stationary (OS)

- Each PE accumulates one output value for the whole computation; weights and activations both stream through it.
- Best when: you want to minimize accumulator read/write traffic (INT32 partial sums are the widest, most expensive data to move).
- Cost: weights must be re-streamed for every new output position, higher weight bandwidth.

Row-Stationary (RS) — used in MIT's Eyeriss

- A hybrid: each PE row is responsible for one 1D convolution "row" of computation, reusing both weights and activations locally within the row, only sending partial sums out.
- Best overall energy efficiency across a range of layer shapes, but the control logic is significantly more complex.

3.3 Why Weight-Stationary Is Right for TinyNPU

For this workload — small spatial resolution (relative to array size), INT8, FPGA-first prototype — weight-stationary is the pragmatic choice for these concrete reasons:

1. Simplicity of control. WS has the simplest load/compute/drain cycle: load weights into the array once, then stream activations for many cycles before reloading. This maps directly onto a simple FSM, which is much easier to verify with cocotb against a Python reference model than OS or RS.
2. INT32 accumulator movement is the most expensive thing you can do. WS keeps the weight stationary and lets the activation stream, meaning the wide INT32 partial sum only moves along one axis (typically vertically through the array) rather than being read/written back to memory every cycle.
3. DSP48 mapping is efficient. Xilinx's DSP48E1 slice has a built-in pre-adder and accumulator register, which naturally implements the "hold weight, stream activation, accumulate downward" pattern of WS with minimal extra logic around each DSP.

3.4 How Data Flows Through a WS Systolic Array, Cycle by Cycle

Picture a 4x4 array computing $Y = W @ X$ (matrix-matrix form, which is how conv is expressed once you apply im2col, see Section 5):

```
Cycle 0:  Weights loaded into all 16 PEs (one weight per PE, held for the whole layer)

Cycle 1:  X[0][0] enters PE(0,0) from the left
          PE(0,0) computes W(0,0)*X[0][0], passes X[0][0] right to PE(0,1) next cycle
          PE(0,0) passes partial sum down to PE(1,0)

Cycle 2:  X[0][1] enters PE(0,0), X[0][0] arrives at PE(0,1)
          PE(0,0) computes new product, PE(0,1) computes W(0,1)*X[0][0]
          Partial sums accumulate downward through the column

...

Cycle N:  Fully "primed" — every PE is doing useful work every cycle
          Output partial sums drain out the bottom row into the accumulator buffer
```

The "priming" and "draining" phases (filling and emptying the pipeline) are overhead — for an $N \times N$ array this costs roughly $2N$ extra cycles per layer. For small arrays (8x8) and layers with many output positions, this overhead is a small fraction of total cycles; it becomes proportionally worse for very small layers, a real consideration for the last (classifier) layers of the network.

3.5 Systolic Array Sizing Trade-off

Array size	DSPs used (XC7S50 has 120)	Pros	Cons
8x8 (64 PEs)	64 (+ headroom for accum/requant logic)	Comfortable DSP margin, easier timing closure, simpler to verify	Lower peak throughput
8x16 (128 PEs)	128 — exceeds available 120 DSPs unless using fabric-logic (LUT) multipliers for some PEs	Higher throughput	Tight/impossible DSP budget on XC7S50, harder timing closure, less margin for other logic
16x16 (256 PEs)	256 — not feasible without INT8 DSP-packing (2 MACs/DSP)	High throughput	Requires DSP packing techniques, likely won't close timing comfortably alongside buffers/DMA/AXI logic

Recommendation stands: 8x8 to start. It leaves ~55 DSPs for the accumulator, requantization multiplier, and auxiliary datapaths, and gives a design that closes timing without heroics — important since this is a learning/verification project, not a tape-out with a schedule that forces pushing density.

4. Memory Hierarchy

4.1 The Three Tiers You're Actually Designing

4. Weight Buffer — holds the current layer's filter weights, loaded once via DMA before compute begins, held stationary for the WS array.
5. Activation Buffer — holds a tile of the input feature map (not the whole thing — SRAM is far too small for that at even modest resolutions), typically double-buffered (ping-pong) so DMA can load the next tile while the array computes on the current one.
6. Output/Accumulator Buffer — holds INT32 partial sums until a full output tile is computed, then requantized to INT8 and written out (or kept on-chip if it's an intermediate layer, avoiding a DRAM round-trip entirely).

4.2 Sizing the Buffers: This Needs Real Numbers, Not Guesses

This is exactly why Phase 2 (Python reference model) must produce concrete layer dimensions before we size any SRAM in Phase 1's final numbers. The sizing formula for a weight buffer tile is:

```
weight_buffer_bytes = K_h * K_w * C_in_tile * C_out_tile * 1 byte (INT8)
```

For an activation buffer tile (with padding halo for convolution):

```
act_buffer_bytes = (tile_h + K_h - 1) * (tile_w + K_w - 1) * C_in_tile * 1 byte
```

Double it for ping-pong. On XC7S50's ~1.35 Mb (≈ 168 KB) of BRAM, budget roughly: 40% weight buffer, 40% activation buffer, 20% accumulator/output staging — a starting split, to be refined once Phase 2 gives us real layer shapes.

4.3 Double Buffering (Ping-Pong) — Why It's Non-Negotiable

Without ping-pong buffering, the compute array stalls every time DMA needs to bring in new data — the array sits idle while waiting for memory. With ping-pong:

Buffer A: [being computed on by array] (swap roles every tile)	Buffer B: [being loaded by DMA]
---	---------------------------------

This is the single highest-leverage architectural technique for keeping the systolic array's utilization high, and it's why the Phase 1 block diagram shows the Activation Buffer as a distinct module from the DMA engine — that separation is what lets them run concurrently.

4.4 On-Chip vs. Off-Chip: The Case for Keeping Everything On-Chip

Since the final network (projectile detection/classification at small input resolutions per-tile) is likely a genuinely tiny model — a few hundred KB to low single-digit MB of weights — a real possibility is that the entire network's weights fit in on-chip BRAM simultaneously, eliminating DRAM traffic (and its $\sim 200\times$ energy cost) almost entirely except for the initial weight load and final result output. Worth confirming numerically once Phase 2 gives us the actual parameter count — if it holds, it meaningfully simplifies the DMA/AXI design and is a legitimate, publishable architectural highlight of the project.

5. From Convolution to Matrix Multiply: im2col

Your systolic array natively does matrix multiplication ($Y = W @ X$). Convolution isn't naturally a matmul — it's a sliding-window operation. The standard trick (used in essentially every accelerator, from Caffe's original CPU implementation onward) is im2col (image-to-column): unroll every sliding window position of the input into a column of a matrix, so convolution becomes one big matmul.

```
Input feature map: H x W x C_in
Filter: K_h x K_w x C_in x C_out

im2col matrix: (K_h * K_w * C_in) rows x (H_out * W_out) columns
Weight matrix: C_out rows x (K_h * K_w * C_in) columns

Output = Weight_matrix @ im2col_matrix → C_out x (H_out * W_out), reshape to H_out
x W_out x C_out
```

The catch: im2col duplicates input data (each pixel appears in multiple overlapping windows), which costs extra memory bandwidth/storage. In software (NumPy) this is usually fine; in hardware, most real accelerators implement a "virtual" or "on-the-fly" im2col — the address generator in the SRAM controller computes the right overlapping read addresses on each cycle rather than physically duplicating data into a separate buffer. This belongs in the SRAM Controller module (Phase 3, item 6), and its correctness is best verified against NumPy's `as_strided`-based im2col in the Python reference model, since that's the golden function to compare against.

Depthwise convolution is a special case where $C_{in} = C_{out}$ and each output channel only depends on one input channel (no cross-channel summation) — this changes the systolic array's data reuse pattern significantly. We'll design this as a separate, simpler streaming mode in the PE array (bypass most of the array, or use a dedicated depthwise path) rather than force-fitting it into the WS matmul dataflow, since forcing it in wastes most of the array's PEs.

6. BatchNorm Fusion

Batch normalization, at inference time, reduces to a per-channel affine transform:

$$\begin{aligned} y &= \text{gamma} * (x - \text{mean}) / \text{sqrt}(\text{var} + \text{eps}) + \text{beta} \\ &= A * x + B \end{aligned}$$

where $A = \text{gamma}/\text{sqrt}(\text{var}+\text{eps})$ and $B = \text{beta} - \text{mean}*A$ are precomputed offline (in the Python reference model / quantization tool, not in hardware). Since a preceding conv layer computes $x = W @ \text{input}$, and BatchNorm is just $A*x + B$, you can algebraically fold it into the conv weights and bias:

$$\begin{aligned} W_{\text{fused}} &= A * W \\ \text{bias}_{\text{fused}} &= B \quad (\text{per output channel}) \end{aligned}$$

This means BatchNorm costs zero extra hardware if fused during the offline quantization/compilation step (exactly what TFLite, ONNX Runtime, and every serious inference compiler does). The RTL never needs a "BatchNorm module" — just a per-channel bias-add in the requantization/activation stage, which is needed anyway. Document this clearly in the architecture doc: BatchNorm fusion is a compile-time transformation in the Python reference model, not an RTL feature.

7. AXI Protocols — What You Actually Need

You already know RTL; here's the AXI-specific mental model for an accelerator context.

7.1 AXI4-Lite (Control Path)

Simple register read/write, no bursts, one outstanding transaction. Used for:

- Configuration registers (layer dimensions, array mode, start/stop control)
- Status registers (done flags, error flags, cycle counters for perf measurement)

This is the easy one — essentially a memory-mapped register file with a standard handshake (AWVALID/AWREADY, WVALID/WREADY, BVALID/BREADY for writes; ARVALID/ARREADY, RVALID/RREADY for reads).

7.2 AXI4-Stream (Data Path)

No addresses — just a continuous handshake (TVALID/TREADY/TDATA, optionally TLAST to mark frame/packet boundaries, TKEEP/TSTRB for byte validity). This is the natural interface for streaming pixel data in from the (future) sensor / DMA, and streaming results out.

Key design point for backpressure: the systolic array's compute rate must be respected by the AXI-Stream interface — if the array can't accept new activation data (e.g., mid-drain of a previous tile), it must deassert TREADY, and the upstream DMA/sensor interface must honor that and stall. Getting

backpressure right end-to-end (DMA $\leftrightarrow$ Activation Buffer $\leftrightarrow$ Systolic Array) is tricky to verify and is exactly what randomized cocotb testbenches are good at catching — random valid/ready toggling on both sides of every AXI-Stream interface should be part of the Phase 4 verification plan.

7.3 Full AXI4 (Memory Path, DMA-facing)

Adds burst support (AWLEN, ARLEN), out-of-order completion via IDs, and wider transaction sizes — this is what the DMA engine uses to talk to external/system memory (or a memory-mapped test harness standing in for DRAM in simulation, since there's no board). For Vivado-only verification, this can be driven against Xilinx's AXI VIP (Verification IP) or a simple behavioral BFM (bus functional model) memory model in the testbench — worth deciding which now, since it affects Phase 3/4 testbench architecture.

8. Controller FSM and Instruction Decoder — Design Philosophy

You don't need (and shouldn't build) a general-purpose instruction set for a first accelerator. The realistic, industry-standard approach for a small NPU is a configuration-descriptor model, not a von-Neumann instruction stream:

```
"Instruction" = a layer descriptor register set:
- layer_type (conv / depthwise / pointwise / fc / pool / elementwise)
- input dimensions (H, W, C_in)
- output dimensions (H_out, W_out, C_out)
- kernel size, stride, padding
- weight_buffer_base_addr, act_buffer_base_addr, output_addr
- activation_function (none / relu / relu6)
- quantization params (multiplier, shift, zero-points if used)
```

The host (or a simple sequencer if the network's whole layer sequence is resident on-chip) writes one descriptor per layer via AXI4-Lite, the Controller FSM decodes it and drives the SRAM controller / systolic array / activation unit through the load $\rightarrow$ compute $\rightarrow$ drain sequence for that layer, then either interrupts the host or auto-advances to the next descriptor if a small descriptor queue/table in on-chip memory is implemented (a nice Phase 6 optimization once the single-layer path is proven).

This is simpler to verify (each descriptor field maps directly to a testable configuration in the Python reference model) and is exactly how real small NPUs (e.g., ARM Ethos-U's command stream, or simplified academic accelerators like DianNao) structure control — a stream of layer descriptors, not a general ISA with branches/loops in hardware.

9. Verification Methodology (Vivado + cocotb/Verilator, No Board)

9.1 The Golden-Model Comparison Loop

1. Python reference model (Phase 2) generates:
 - random or real test input tensors
 - expected output tensors (bit-exact INT8 arithmetic, matching planned hardware precision)
 - intermediate values (per-layer, per-tile) for granular debug
2. RTL testbench (cocotb, driving either Verilator or Vivado's xsim):

- drives the same inputs into the DUT via AXI-Stream/AXI-Lite BFM
- captures DUT outputs
- compares bit-exact against the Python golden values

3. Mismatch → dump waveform (.vcd/.wdb), inspect in Vivado's waveform viewer, root-cause

Critical point: the Python reference model must do integer-exact arithmetic, not floating-point approximation, or "verification" against it is meaningless — Phase 2 must actually implement INT8 multiply / INT32 accumulate / fixed-point requantize using NumPy's integer dtypes (np.int8, np.int32) with explicit overflow/saturation handling, not just float32 matmuls rounded afterward. A float-based "reference" model will pass most tests and then mysteriously disagree with hardware on edge cases (saturation, rounding direction) — exactly the class of bug that's expensive to find late.

9.2 Test Tiers (Weakest to Strongest Signal)

7. Directed unit tests per module (e.g., single PE: known weight, known input, check known output) — catches basic logic bugs, fast to write, low coverage.
8. Randomized tests (cocotb + a constrained-random stimulus generator) — random weight/activation values within valid quantized ranges, compared against the Python golden function. Catches corner cases directed tests miss.
9. Layer-level tests — a full conv/pool/fc layer with realistic dimensions, verified end-to-end against Phase 2's per-layer golden output.
10. Network-level test — the full model, real (or synthetic sensor-simulator) input image through, compared against Python's final classification/detection output.
11. Assertions (SVA) — protocol-level invariants that should never be violated regardless of data values: AXI handshake rules, no accumulator overflow past INT32, no simultaneous read/write conflicts on ping-pong buffers, FSM never entering an undefined state.

9.3 Coverage — What "Done" Means for a Module

A module isn't verified because "the testbench passed once." Track:

- Functional coverage: have you exercised every layer type, every valid dimension range, every corner (1x1 kernel, largest kernel, minimum/maximum channel counts)?
- Code coverage (Vivado can report this): every branch of every FSM, every case arm, has been hit.

This matters most for the Controller FSM and Instruction Decoder — these are the modules most likely to have "forgot a state transition" bugs that directed tests with typical inputs won't find.

10. Vivado Flow Without a Board — What Actually Changes

10.1 Project Setup

Set the target part explicitly (xc7s50csga324-1 for the Spartan-7 XC7S50) even with no board — this is what makes synthesis, implementation, and STA numbers meaningful and comparable to a real deployment later. A minimal or synthetic XDC (constraints file) is still needed: at minimum a clock

constraint (create_clock) so Vivado can run timing analysis; I/O constraints can be left generic/unconstrained since there's no physical pin mapping to honor.

10.2 Simulation

Two realistic paths, pick one as primary:

- Vivado's xsim — fully integrated, good waveform viewer, straightforward if already comfortable in the Vivado GUI, but slower for large random-test suites and less common in industry-standard open verification flows.
- Verilator + cocotb — much faster simulation (compiles to C++), Python-native testbenches (natural fit for comparing against the NumPy golden model), industry-standard for this kind of verification flow, but Verilator doesn't support all SystemVerilog constructs and waveform viewing requires exporting to .vcd/.fst and opening in a separate viewer (GTKWave, or importable into Vivado's viewer).

Recommendation: cocotb+Verilator as the primary automated verification loop (fast, scriptable, integrates naturally with Python golden-model comparison), with Vivado xsim reserved for final visual waveform inspection of specific modules, and for anything needing Xilinx-specific IP (e.g., a directly-instantiated Xilinx BRAM primitive rather than inferred SRAM).

10.3 Synthesis, Implementation, Reports

Once RTL is verified functionally:

- Synthesis → check for unexpected inferred latches, resource estimates
- Implementation (place & route) → real timing closure numbers, real resource utilization (DSP/BRAM/LUT/FF counts) from the actual physical part
- Timing report → gives real Fmax for the design, which directly answers the Phase 5 "throughput" question — $\text{cycles-per-inference (from simulation)} \times \text{achievable clock period (from STA)} = \text{real projected latency, without ever touching hardware}$
- Power report (Vivado Power Analyzer) → needs a realistic switching-activity file (.saif or .vcd from simulation with representative test vectors) to be meaningful — use real/representative image data from the Python sensor simulator when generating the activity file for this step.

11. Vocabulary Reference

Term	Meaning
MAC	Multiply-accumulate — the fundamental operation
PE	Processing Element — one MAC unit + local control/registers in the array
GEMM	General Matrix Multiply — the core operation most accelerators are built around
im2col	Image-to-column — the transform that turns convolution into GEMM
Roofline model	Performance model plotting achievable throughput vs. arithmetic intensity (ops per byte moved)

Arithmetic intensity	MACs per byte of data moved — high = compute-bound, low = memory-bound
QAT	Quantization-aware training — training with simulated quantization noise for INT8 robustness
Zero-skipping	Don't spend a cycle multiplying by zero (common after ReLU, or in sparse networks)
Layer fusion	Combining multiple ops (e.g., conv+BN+ReLU) into one hardware pass to avoid extra memory round-trips
Priming/draining	Pipeline fill/empty overhead cycles at the start/end of a systolic array computation
Ping-pong buffering	Two buffers alternating between "being computed on" and "being loaded"
WS / OS / RS	Weight-, Output-, Row-stationary — the three canonical systolic dataflow classes
Requantization	Converting INT32 accumulator values back to INT8 via integer multiply + shift
Array utilization	Fraction of cycles where every PE does useful (non-priming/draining/stall) work

12. Suggested Reading

These aren't required before continuing, but they're the reference points worth knowing for future trade-off discussions — at least skim the abstracts/architecture figures:

- Eyeriss (MIT, Chen et al.) — the canonical row-stationary dataflow paper; excellent diagrams of the energy-per-data-movement argument from Section 1.
- Google TPU v1 (Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit") — the reference weight-stationary systolic array design at large scale.
- DianNao / ShiDianNao (Chen et al.) — small-footprint accelerator papers, closer in spirit and scale to TinyNPU than TPU/Eyeriss.
- ARM Ethos-U (product architecture docs) — a real shipping small-edge NPU; useful for seeing a descriptor/command-stream control model in an actual product.
- gemmlowp documentation (Google) — the clearest public explanation of the integer-only quantization math in Section 2.

Next Step

With this foundation in place, the Phase 1 architecture document (with real throughput, power, and resource estimates) can be finalized once the following are confirmed:

12. The array size decision (8x8 recommended)
13. INT4 timing (deferred to Phase 6, recommended)
14. Whether to proceed straight to Phase 2 (Python reference model) next, since several sections above (memory sizing, verification golden-model correctness) depend on real layer dimensions from a concrete network architecture.